

GUIDE ET SUPPORT DE COURS FULL STACK

Conception et développement d'applications web modernes de A à Z

Niveau : Débutant à Intermédiaire

Technologies : MERN Stack (React, Node, Mongo)

Format : Cours & Pratique

1. Introduction au Développement Full Stack

Le développeur **Full Stack** est un profil polyvalent capable de concevoir, développer et maintenir toutes les couches d'une application informatique, de l'interface utilisateur (**Frontend**) jusqu'à la logique métier et le stockage des données (**Backend & Base de données**).

Architecture Client-Serveur

- **Frontend (Client) :** S'exécute dans le navigateur de l'utilisateur. Il gère l'affichage, le style, le rendu visuel et l'interaction directe (HTML5, CSS3, JavaScript / React).
- **Backend (Serveur) :** S'exécute sur le serveur distant. Il traite la logique métier, la sécurité, l'authentification et communique avec la base de données (Node.js, Express, Python, Java).
- **Base de données :** Stocke et organise les informations de manière persistante (MongoDB, PostgreSQL, MySQL).
- **API REST / RESTful :** Interface de communication standardisée entre le client et le serveur via le protocole HTTP.

2. Module 1 : Les Fondations du Web (HTML5 & CSS3)

HTML structure le contenu web tandis que CSS définit la présentation visuelle et l'agencement adaptatif (responsive design).

Code HTML5 & CSS3 Responsive

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mon Application Full Stack</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="header">
    <h1>Bienvenue sur ma plateforme</h1>
  </header>
  <main class="container">
    <div class="card">
      <h2>Produit Vedette</h2>
      <p>Découvrez nos solutions informatiques sur mesure.</p>
    </div>
  </main>
</body>
</html>
```

3. Module 2 : JavaScript Moderne (ES6+)

JavaScript est le langage incontournable du développement web. La norme ES6+ a introduit de nombreuses fonctionnalités modernes indispensables :

Exemple : Syntaxe moderne et Asynchronisme (Promises & Async/Await)

```
// 1. Fonctions fléchées et Déstructuration
const user = { name: 'Alice', role: 'Développeuse', age: 28 };
const { name, role } = user;
console.log(`${name} est ${role}`);

// 2. Programmation asynchrone avec Fetch & Async/Await
async function fetchProducts() {
  try {
    const response = await fetch('https://api.example.com/products');
    if (!response.ok) throw new Error('Erreur réseau');
    const data = await response.json();
    return data;
  } catch (error) {
    console.error('Échec du chargement :', error.message);
  }
}
```

4. Module 3 : Frontend Dynamique avec React.js

React est une bibliothèque JavaScript créée par Meta basée sur les composants autonomes et le DOM virtuel.

Exemple : Composant React avec Hooks (useState, useEffect)

```
import React, { useState, useEffect } from 'react';

function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('http://localhost:5000/api/users')
      .then((res) => res.json())
      .then((data) => {
        setUsers(data);
        setLoading(false);
      });
  }, []);

  if (loading) return <p>Chargement des données...</p>;

  return (
    <div>
      <h3>Liste des utilisateurs</h3>
      <ul>
        {users.map((u) => (
          <li key={u.id}>{u.name} - {u.email}</li>
        ))}
      </ul>
    </div>
  );
}

export default UserList;
```

5. Module 4 : Backend & API REST avec Node.js & Express

Node.js permet d'exécuter JavaScript côté serveur. Express.js simplifie la création de routes HTTP pour construire une API RESTful robuste.

Exemple : Serveur Express et Endpoints REST API

```
const express = require('express');
const app = express();
app.use(express.json());

let todos = [
  { id: 1, task: 'Apprendre Node.js', completed: false }
];

// Route GET : Récupérer les tâches
app.get('/api/todos', (req, res) => {
  res.json(todos);
});

// Route POST : Ajouter une nouvelle tâche
app.post('/api/todos', (req, res) => {
  const newTodo = { id: Date.now(), task: req.body.task, completed: false };
  todos.push(newTodo);
  res.status(201).json(newTodo);
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Serveur démarré sur le port ${PORT}`));
```

6. Module 5 : Base de Données (MongoDB & Mongoose)

MongoDB est une base de données NoSQL orientée documents JSON/BSON. Mongoose fournit une modélisation d'objets élégante pour Node.js.

Exemple : Schéma Mongoose et Modèle Utilisateur

```
const mongoose = require('mongoose');

const UserSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  email: { type: String, required: true },
  password: { type: String, required: true },
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('User', UserSchema);
```

7. Projets Pratiques à Réaliser

Pour maîtriser l'ensemble des compétences Full Stack, il est essentiel de réaliser des projets concrets de complexité croissante.

Projet 1 - Débutant

Gestionnaire de Tâches Interactif (Todo App)

Objectif : Créer une application classique de suivi de tâches individuelles.

- **Stack :** React, Node.js, Express, MongoDB.
- **Fonctionnalités :**
 - Affichage de la liste des tâches à accomplir.
 - Ajout, modification et suppression de tâches (CRUD).
 - Filtrage par statut (Toutes, En cours, Terminées).
- **Livrable attendu :** Interface propre en React connectée à une API Express persistante.

Projet 2 - Intermédiaire

Plateforme de Blog Multi-utilisateurs avec Authentification

Objectif : Concevoir un système complet de gestion de contenu sécurisé.

- **Stack :** MERN Stack (MongoDB, Express, React, Node), JWT (JSON Web Tokens).
- **Fonctionnalités :**
 - Inscription et connexion sécurisées (hachage de mot de passe avec `bcrypt`).
 - Authentification basée sur les JSON Web Tokens (JWT).
 - Publication, édition et suppression d'articles réservées à l'auteur connecté.
 - Espace commentaires interactif sous chaque article.

Projet 3 - Avancé

Mini-Plateforme E-commerce (Boutique en Ligne)

Objectif : Développer une application d'e-commerce prête pour la production.

- **Stack :** React, Node.js, Express, MongoDB, Tailwind CSS, Stripe (Paiement).
- **Fonctionnalités :**
 - Catalogue de produits avec recherche, filtrage par catégorie et pagination.
 - Panier d'achat dynamique avec gestion de la quantité.
 - Passerelle de paiement intégrée (Stripe / PayPal en Sandbox).
 - Tableau de bord Administrateur pour gérer le stock et voir les commandes client.

8. Feuille de Route & Bonnes Pratiques

Domaine	Outils & Concepts Clés	Recommandation
Versionning	Git, GitHub / GitLab	Commiter régulièrement avec des messages explicites.
Sécurité	HTTPS, CORS, Helmet, JWT, bcrypt	Ne jamais stocker de mots de passe ou clés en clair.
Déploiement	Vercel / Netlify (Front), Render / Railway (Back)	Mettre en place un pipeline CI/CD automatisé.